

PE02 - Infraestructura WordPress Multi-máquina

Peso: 35% de la nota

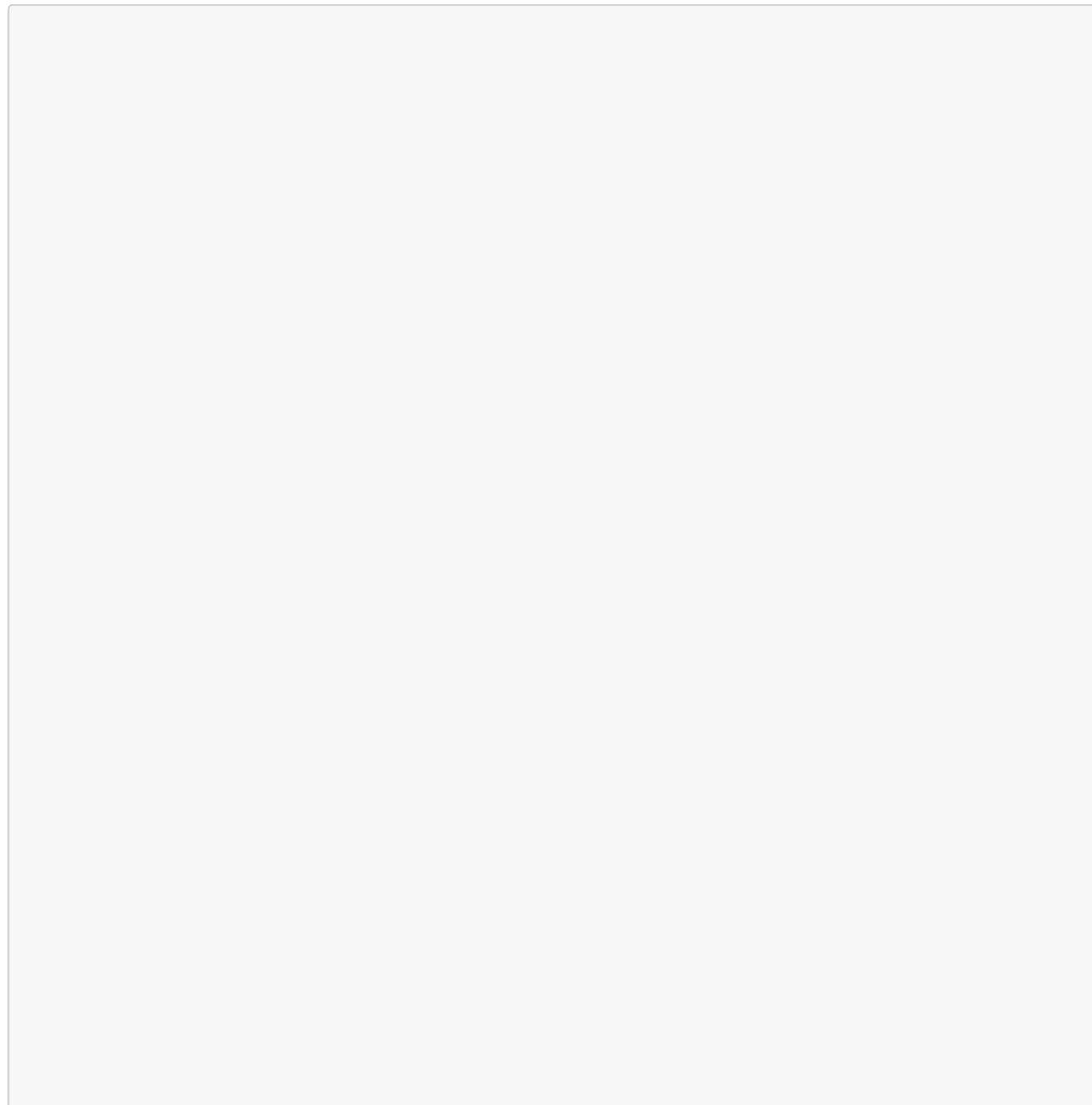
Duración: 3-4 horas

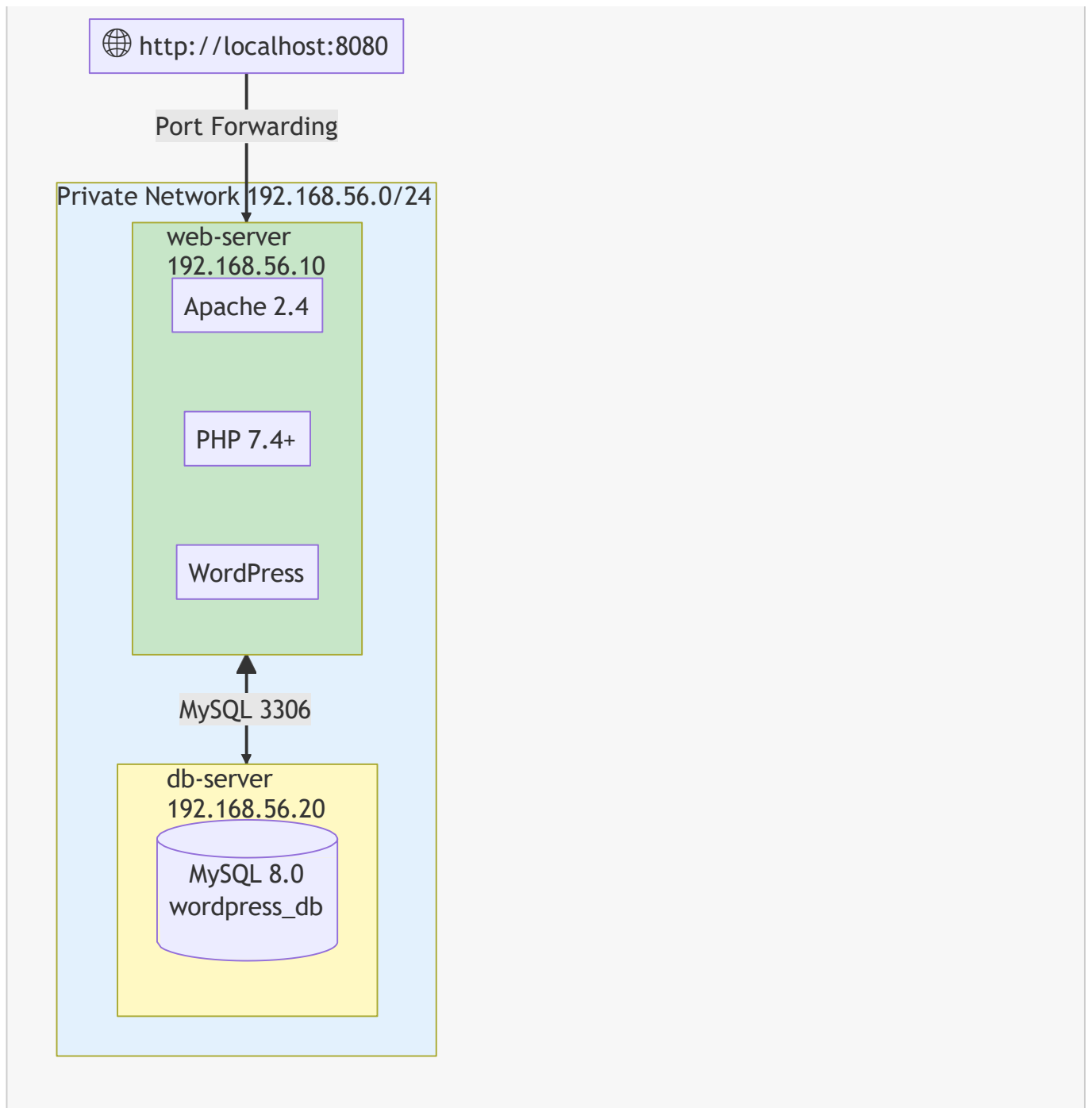
Dificultad: ☆ ☆ ☆ Media-Alta

Objetivo

Crear una infraestructura WordPress profesional con separación de servicios: un servidor web con Apache y PHP, y un servidor de base de datos MySQL separado. Ambos comunicándose a través de red privada.

Arquitectura requerida





Requisitos mínimos (6 puntos)

- 1 Dos VMs funcionando (1 punto)
 - o web-server con Apache + PHP

```
scripts > $ install-web.sh
```

```
1  #!/bin/bash
2  set -e
3  export DEBIAN_FRONTEND=noninteractive
4
5  echo "=== Instalando Apache ==="
6  apt-get install -y apache2
7
8  echo "=== Instalando PHP y extensiones ==="
9  apt-get install -y php libapache2-mod-php php-mysql php-curl php-gd \
10 | php-mbstring php-xml php-xmlrpc php-zip php-intl php-opcache
11
12 echo "=== Habilitando módulos Apache ==="
13 a2enmod rewrite
14
15 echo "=== Descargando WordPress ==="
16 cd /tmp
17 wget -q https://wordpress.org/latest.tar.gz
18 tar -xzf latest.tar.gz
19
20 echo "=== Instalando WordPress ==="
21 rm -rf /var/www/html/*
22 cp -r wordpress/* /var/www/html/
23
24 echo "=== Configurando permisos iniciales ==="
25 chown -R www-data:www-data /var/www/html
26 find /var/www/html/ -type d -exec chmod 755 {} \;
27 find /var/www/html/ -type f -exec chmod 644 {} \;
28
29 echo "=== Apache y PHP instalados ==="
```

1

- db-server con MySQL

```
scripts > $ install-db.sh
```

```

1  #!/bin/bash
2  set -e
3  export DEBIAN_FRONTEND=noninteractive
4
5  echo "=== Instalando MySQL Server ==="
6  apt-get install -y mysql-server
7
8  echo "=== Configurando MySQL para acceso remoto ==="
9  # Habilitar la escucha en todas las interfaces de red (0.0.0.0)
10 sed -i 's/bind-address.*/bind-address = 0.0.0.0/' /etc/mysql/mysql.conf.d/mysqld.cnf
11
12 systemctl restart mysql
13
14 echo "=== Creando base de datos y usuario ==="
15 mysql <<EOF
16 CREATE DATABASE IF NOT EXISTS wordpress_db DEFAULT CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
17
18 -- Crear usuario estricto para la subred privada (Seguridad MySQL)
19 CREATE USER IF NOT EXISTS 'wp_user'@'192.168.56.%' IDENTIFIED BY 'wp_secure_pass';
20
21 -- Otorgar permisos específicos a la BD de wordpress
22 GRANT ALL PRIVILEGES ON wordpress_db.* TO 'wp_user'@'192.168.56.%';
23 FLUSH PRIVILEGES;
24
25 -- Verificaciones de control en el log de Vagrant
26 SHOW DATABASES;
27 SELECT User, Host FROM mysql.user;
28 EOF
29
30 echo "=== MySQL configurado correctamente ==="
```

```
}
```

```

C:\Users\Adrian\Documents\Computación en la nube y v
C:\Users\Adrian\Documents\Computación en la nube y v
Current machine states:

```

```

db          running (virtualbox)
web         running (virtualbox)

```

4 Networking correcto (1 punto)

- Red privada entre VMs

```

# === DB SERVER (Primero para que esté listo antes que WordPress) ===
config.vm.define "db" do |db|
  db.vm.hostname = "db-server"
  db.vm.network "private_network", ip: DB_IP

```

- Port forwarding del host al web-server

```
# === WEB SERVER ===
config.vm.define "web", primary: true do |web|
  web.vm.hostname = "web-server"
  web.vm.network "private_network", ip: WEB_IP
  web.vm.network "forwarded_port", guest: 80, host: 8080
```

5 MySQL configurado remotamente (1.5 puntos)

- Escucha en 0.0.0.0

```
echo "=== Configurando MySQL para acceso remoto ==="
# Habilitar la escucha en todas las interfaces de red (0.0.0.0)
sed -i 's/bind-address.*/bind-address = 0.0.0.0/' /etc/mysql/mysql.conf.d/mysqld.cnf
```

- Usuario con permisos desde red privada

-- Otorgar permisos específicos a la BD de wordpress

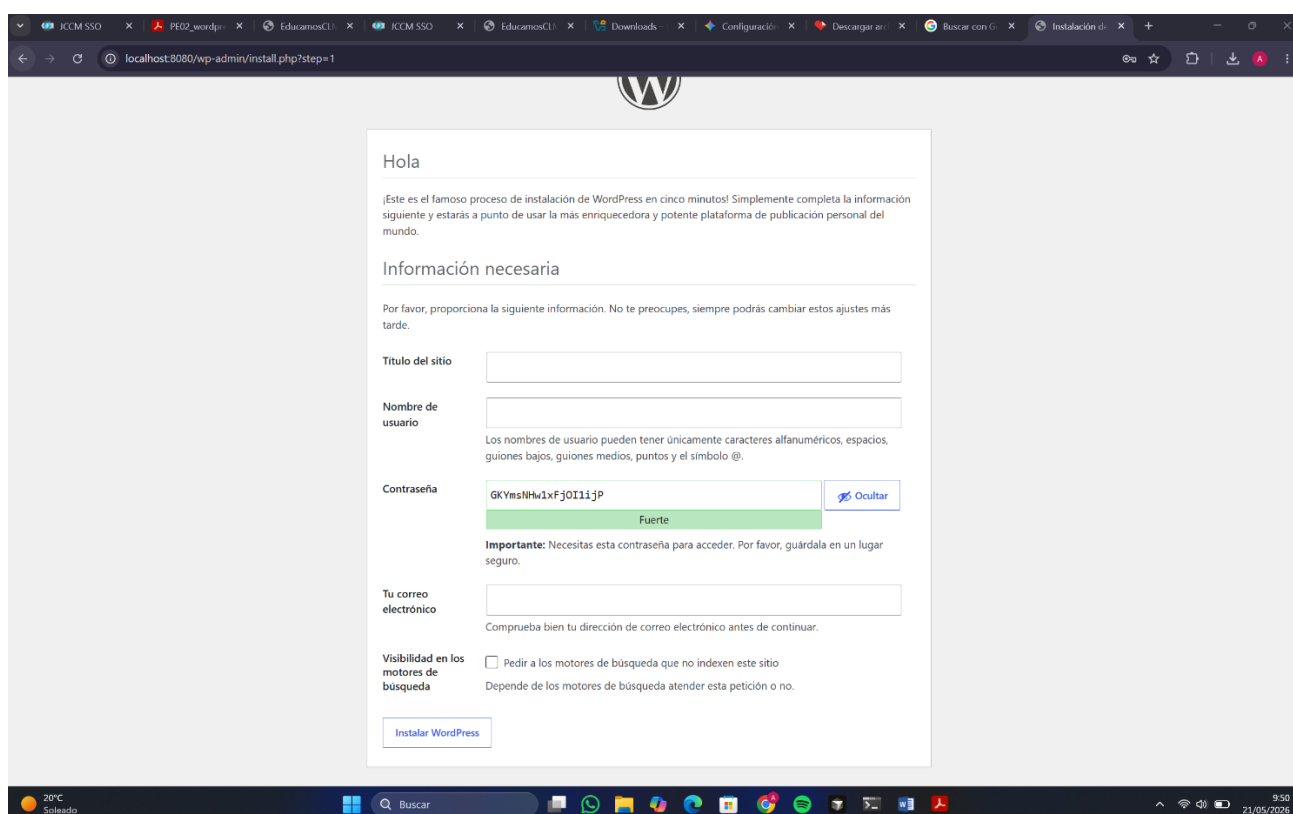
```
GRANT ALL PRIVILEGES ON wordpress_db.* TO 'wp_user'@'192.168.56.%';
FLUSH PRIVILEGES;
```

- Base de datos creada

```
CREATE DATABASE IF NOT EXISTS wordpress_db DEFAULT CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
```

6 WordPress instalado (1.5 puntos)

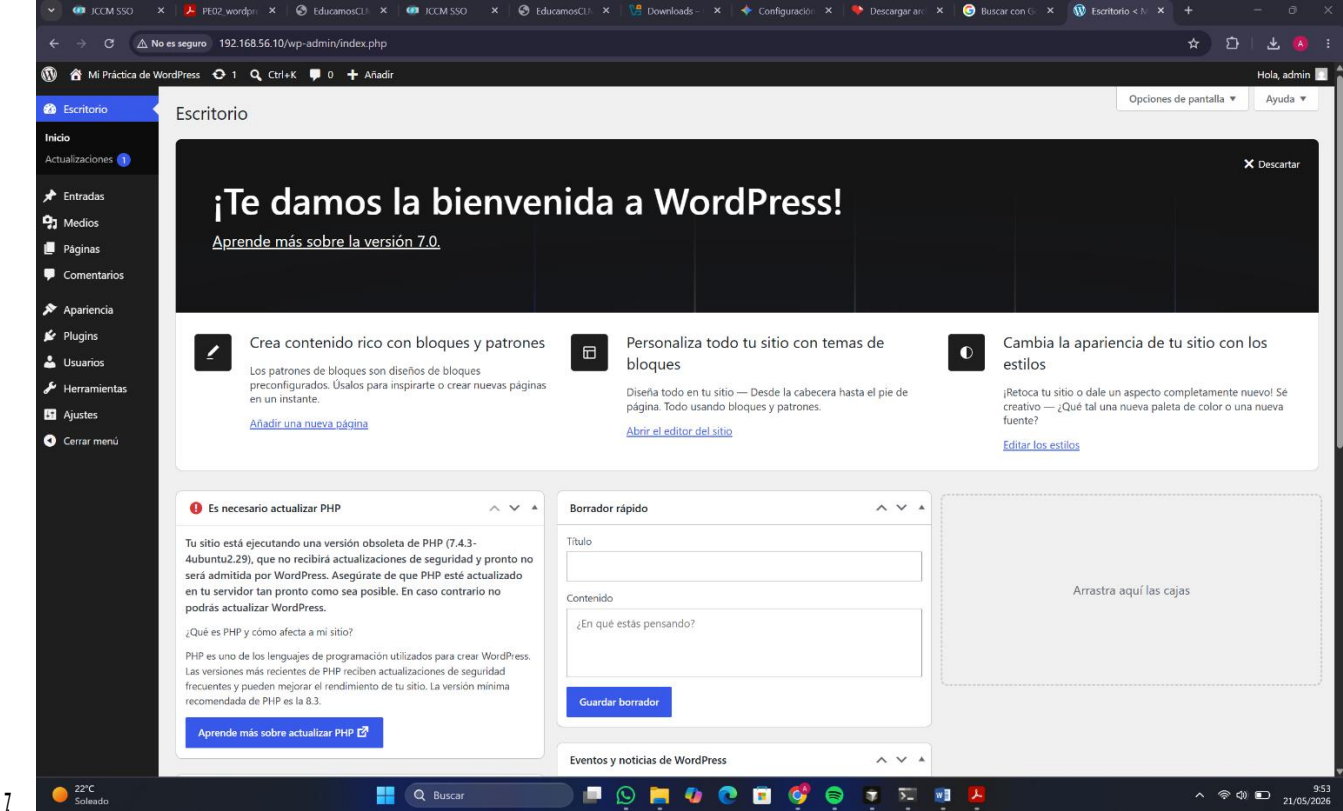
- Descargado e instalado



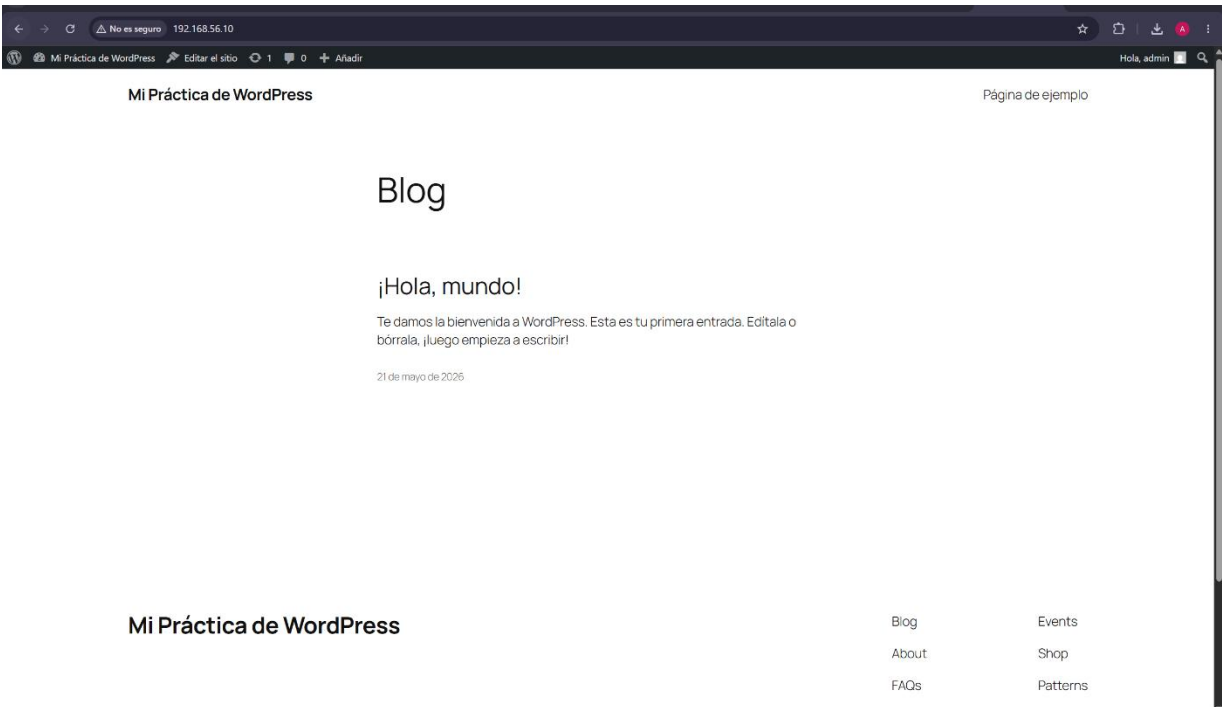
```
echo "=== Descargando WordPress ==="
cd /tmp
wget -q https://wordpress.org/latest.tar.gz
tar -xzf latest.tar.gz
```

```
echo "=== Instalando WordPress ==="
rm -rf /var/www/html/*
cp -r wordpress/* /var/www/html/
```

o Conectado a BD remota



o Accesible desde navegador



8 Provisioning automatizado (1 punto)

- o Sin intervención manual

```
ots > $ install-web.sh
```

```
1  #!/bin/bash
```

```
2  set -e
```

```
3  export DEBIAN_FRONTEND=noninteractive
```

```
9  1
```

Requisitos avanzados (3 puntos)

0 Resolución de nombres (0.5 puntos)

- o /etc/hosts configurado en ambas VMs

Todos (1) | Administrador (1)

Acciones en lote

Aplicar

Cambiar perfil a...

Cambiar

1 elemento

☐	Nombre de usuario	Nombre	Correo electrónico	Perfil	Entradas
☐	 admin	—	adriandelgadocampos@gmail.com	Administrador	1
☐	Nombre de usuario	Nombre	Correo electrónico	Perfil	Entradas

Acciones en lote

Aplicar

Cambiar perfil a...

Cambiar

1 elemento

1 wp-config.php autoconfigurable (1 punto)

- o Generado automáticamente
- o Credenciales correctas

2 Permisos correctos (0.5 puntos)

- o www-data propietario de WordPress

```
echo "=== Configurando permisos iniciales ==="
chown -R www-data:www-data /var/www/html
find /var/www/html/ -type d -exec chmod 755 {} \;
find /var/www/html/ -type f -exec chmod 644 {} \;
```


8 Scripts organizados (1 punto)

- Scripts separados para cada servicio

- ▼ config
 - ⚙️ wordpress.conf
- ▼ scripts
 - \$ common.sh
 - \$ configure-wordpress.sh
 - \$ install-db.sh
 - \$ install-web.sh
- 📄 Vagrantfile

Requisitos extras (1 punto)

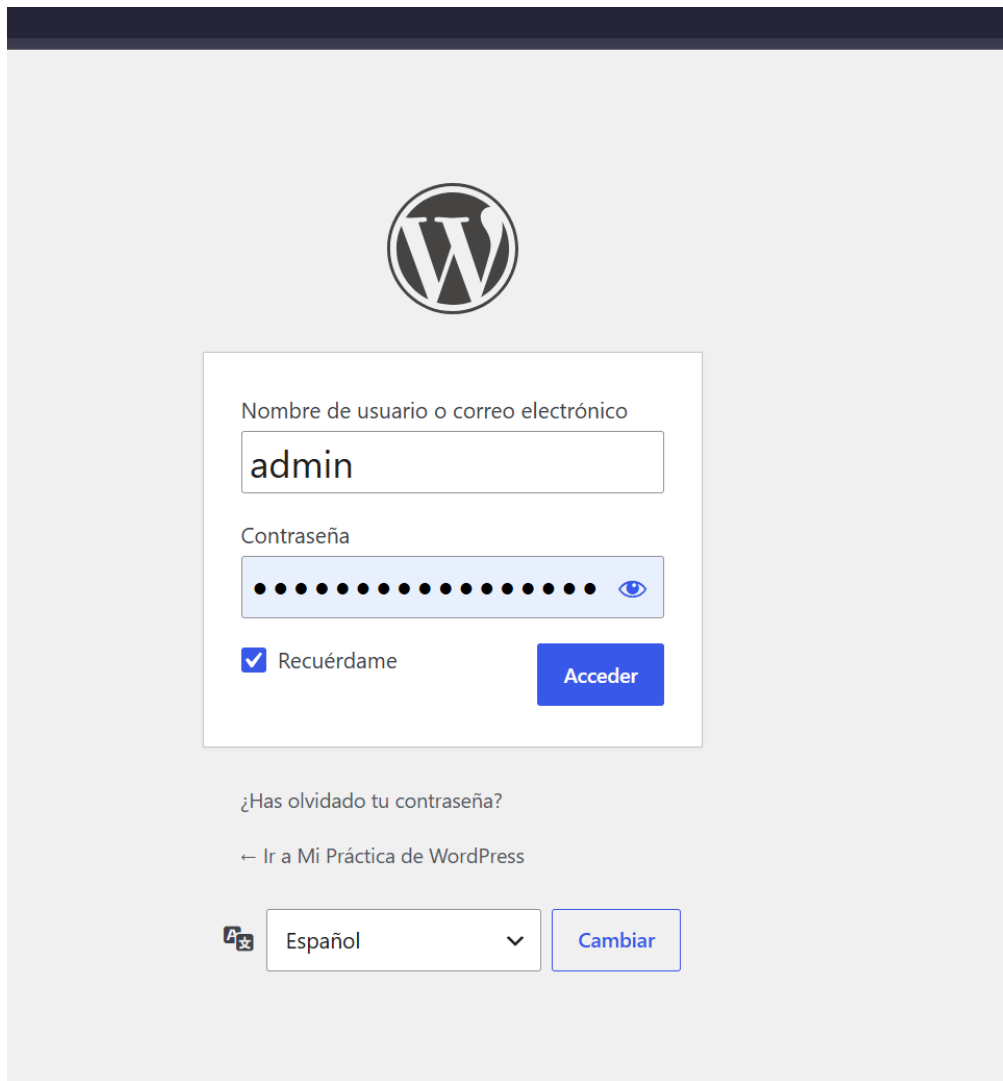
10. Cache de opcodes (0.25 puntos)

- PHP OPcache habilitado

```
6 apt-get install -y apache2
7
8 echo "=== Instalando PHP y extensiones ==="
9 apt-get install -y php libapache2-mod-php php-mysql php-curl php-gd \
10 php-mbstring php-xml php-xmlrpc php-zip php-intl php-opcache
11
```

11. Seguridad MySQL (0.25 puntos)

- ☐ Solo usuario necesario, sin root remoto



12. README profesional (0.5 puntos)

- Documentación completa con diagramas

PE02 - Infraestructura WordPress Multi-máquina

🚀 Descripción

Proyecto de despliegue de una infraestructura WordPress utilizando dos máquinas virtuales:

- **web-server** → Apache + PHP + WordPress
- **db-server** → MySQL 8.0

Ambas máquinas se comunican mediante una red privada configurada con Vagrant y VirtualBox.

🏗️ Arquitectura

HOST → localhost:8080



⚙️ Tecnologías

- Vagrant
- VirtualBox
- Apache 2.4
- PHP 7.4+
- MySQL 8.0
- WordPress

📁 Estructura

```
PE02_wordpress/  
├── Vagrantfile  
├── README.md  
└── scripts/  
    ├── common.sh  
    ├── install-db.sh  
    ├── install-web.sh  
    └── configure-wordpress.sh
```

🚀 Despliegue

Levantar las máquinas

```
vagrant up
```

Verificar estado

```
vagrant status
```

Verificar conectividad

```
vagrant ssh web -c "ping -c 3 db-server"
```

Acceso

Abrir en el navegador:

```
http://localhost:8080
```

Base de datos

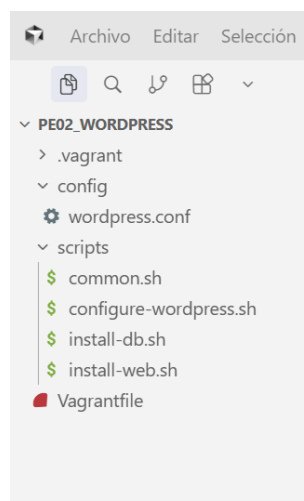
Parámetro	Valor
Base de datos	wordpress_db
Usuario	wp_user
Contraseña	wp_secure_pass

Autor

Delgado Campos

https://github.com/Adri06879/Practicas-Vagrant-2-Asir/tree/main/PE02_wordpress

Estructura del proyecto



README.md con:

- Diagrama de arquitectura
 - Instrucciones paso a paso
 - Credenciales de acceso
 - Capturas de pantalla
- 1 Configuración de virtual host (opcional)

Criterios de evaluación

Aspecto	Puntos	Descripción
Funcionalidad	4	Todo funciona correctamente
Multi-máquina	2	Comunicación correcta entre VMs
Automatización	2	Sin intervención manual
Organización	1	Código limpio y modular
Documentación	1	README completo

Guía de implementación

1. Vagrantfile estructura

```

Vagrantfile
1  # -*- mode: ruby -*-
2  #vi: set ft=ruby:
3
4  SUBNET = "192.168.56"
5  WEB_IP = "#{SUBNET}.10"
6  DB_IP = "#{SUBNET}.20"
7
8  Vagrant.configure("2") do |config|
9    config.vm.box = "ubuntu/focal64"
10
11    # === DB SERVER (Primero para que esté listo antes que WordPress) ===
12    config.vm.define "db" do |db|
13      db.vm.hostname = "db-server"
14      db.vm.network "private_network", ip: DB_IP
15
16      db.vm.provider "virtualbox" do |vb|
17        vb.name = "WP-Database"
18        vb.memory = "2048"
19      end
20
21      db.vm.provision "shell", path: "scripts/common.sh"
22      db.vm.provision "shell", path: "scripts/install-db.sh"
23    end
24
25    # === WEB SERVER ===
26    config.vm.define "web", primary: true do |web|
27      web.vm.hostname = "web-server"
28      web.vm.network "private_network", ip: WEB_IP
29      web.vm.network "forwarded_port", guest: 80, host: 8080
30
31      web.vm.provider "virtualbox" do |vb|
32        vb.name = "WP-WebServer"
33        vb.memory = "1024"
34      end
35
36      web.vm.provision "shell", path: "scripts/common.sh"
37      web.vm.provision "shell", path: "scripts/install-web.sh"
38      web.vm.provision "shell", path: "scripts/configure-wordpress.sh", env: {
39        "DB_HOST" => DB_IP,
40        "DB_NAME" => "wordpress_db",
41        "DB_USER" => "wp_user",
42        "DB_PASS" => "wp_secure_pass"
43      }
44    end
45  end

```

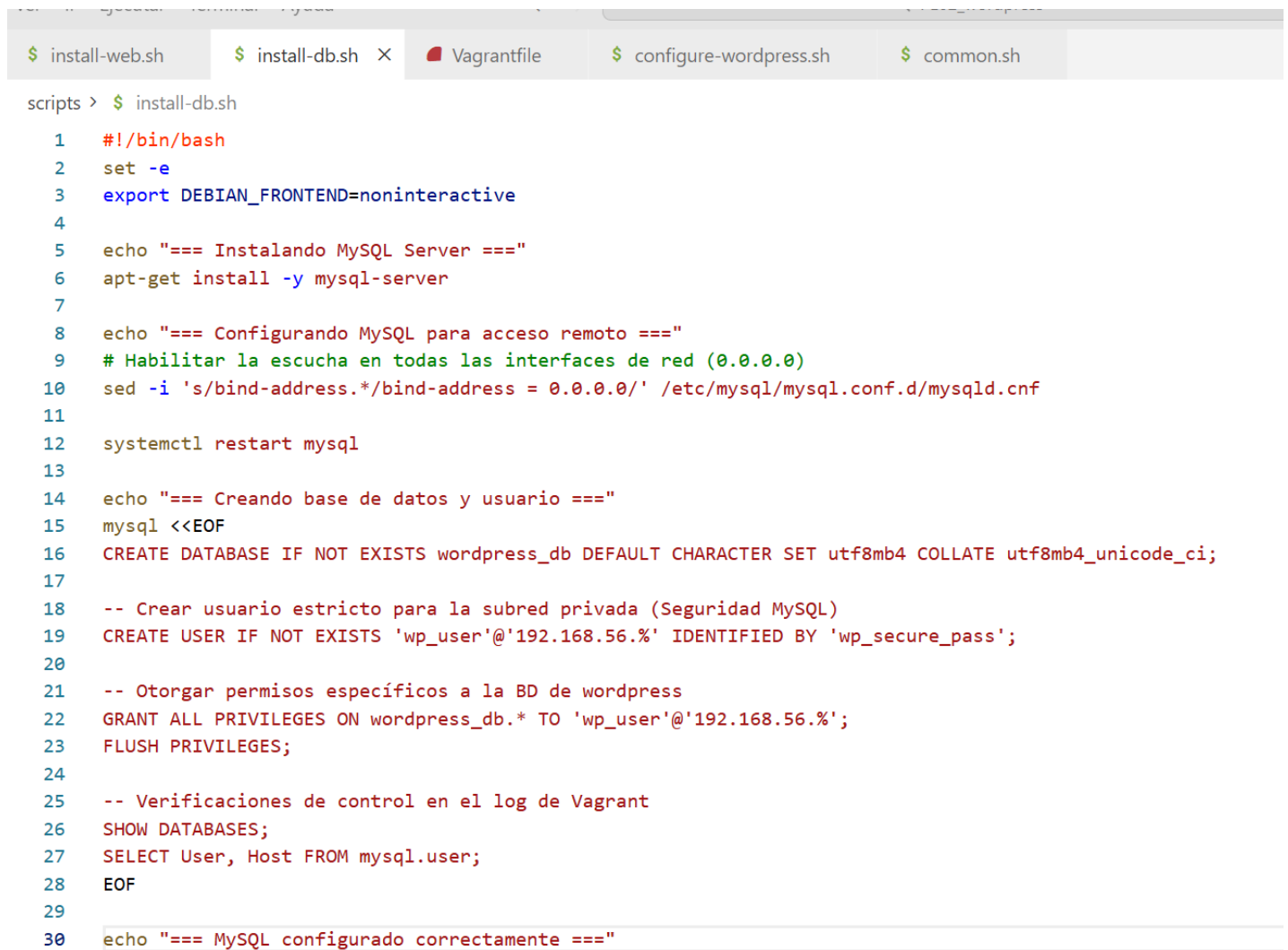
2. Script common.sh

```

$ install-web.sh  $ install-db.sh  Vagrantfile  $ configure-wordpress.sh  $ common.sh x
scripts > $ common.sh
1  #!/bin/bash
2  set -e
3
4  echo "=== Configuración común ==="
5
6  # Actualizar sistema
7  apt-get update
8
9  # Instalar utilidades básicas
10 apt-get install -y vim curl wget net-tools dnsutils
11
12 # Configurar /etc/hosts para resolución de nombres interna
13 cat <<EOF > /etc/hosts
14 127.0.0.1 localhost
15 192.168.56.10 web-server
16 192.168.56.20 db-server
17 EOF
18
19 # Configurar timezone
20 timedatectl set-timezone Europe/Madrid
21
22 echo "=== Configuración común completada ==="
23

```

3. Script install-db.sh



```

$ install-web.sh  $ install-db.sh X  Vagrantfile  $ configure-wordpress.sh  $ common.sh

scripts > $ install-db.sh
1  #!/bin/bash
2  set -e
3  export DEBIAN_FRONTEND=noninteractive
4
5  echo "=== Instalando MySQL Server ==="
6  apt-get install -y mysql-server
7
8  echo "=== Configurando MySQL para acceso remoto ==="
9  # Habilitar la escucha en todas las interfaces de red (0.0.0.0)
10 sed -i 's/bind-address.*/bind-address = 0.0.0.0/' /etc/mysql/mysql.conf.d/mysqld.cnf
11
12 systemctl restart mysql
13
14 echo "=== Creando base de datos y usuario ==="
15 mysql <<EOF
16 CREATE DATABASE IF NOT EXISTS wordpress_db DEFAULT CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
17
18 -- Crear usuario estricto para la subred privada (Seguridad MySQL)
19 CREATE USER IF NOT EXISTS 'wp_user'@'192.168.56.%' IDENTIFIED BY 'wp_secure_pass';
20
21 -- Otorgar permisos específicos a la BD de wordpress
22 GRANT ALL PRIVILEGES ON wordpress_db.* TO 'wp_user'@'192.168.56.%;
23 FLUSH PRIVILEGES;
24
25 -- Verificaciones de control en el log de Vagrant
26 SHOW DATABASES;
27 SELECT User, Host FROM mysql.user;
28 EOF
29
30 echo "=== MySQL configurado correctamente ==="

```

4. Script install-web.sh

\$ install-web.sh × \$ install-db.sh Vagrantfile \$ configure-wordpress.sh \$ common.s

scripts > \$ install-web.sh

```

1  #!/bin/bash
2  set -e
3  export DEBIAN_FRONTEND=noninteractive
4
5  echo "=== Instalando Apache ==="
6  apt-get install -y apache2
7
8  echo "=== Instalando PHP y extensiones ==="
9  apt-get install -y php libapache2-mod-php php-mysql php-curl php-gd \
10 | php-mbstring php-xml php-xmlrpc php-zip php-intl php-opcache
11 |
12 echo "=== Habilitando módulos Apache ==="
13 a2enmod rewrite
14
15 echo "=== Descargando WordPress ==="
16 cd /tmp
17 wget -q https://wordpress.org/latest.tar.gz
18 tar -xzf latest.tar.gz
19
20 echo "=== Instalando WordPress ==="
21 rm -rf /var/www/html/*
22 cp -r wordpress/* /var/www/html/
23
24 echo "=== Configurando permisos iniciales ==="
25 chown -R www-data:www-data /var/www/html
26 find /var/www/html/ -type d -exec chmod 755 {} \;
27 find /var/www/html/ -type f -exec chmod 644 {} \;
28
29 echo "=== Apache y PHP instalados ==="

```

5. Script configure-wordpress.sh


```

$ install-web.sh  $ install-db.sh  Vagrantfile  $ configure-wordpress.sh  X  $ common.sh
scripts > $ configure-wordpress.sh

1  #!/bin/bash
2  set -e
3
4  echo "=== Configurando WordPress ==="
5  cd /var/www/html
6
7  # Crear wp-config.php a partir de la plantilla
8  cp wp-config-sample.php wp-config.php
9
10 # Inyectar credenciales pasadas por el entorno de Vagrant
11 sed -i "s/database_name_here/$DB_NAME/" wp-config.php
12 sed -i "s/username_here/$DB_USER/" wp-config.php
13 sed -i "s/password_here/$DB_PASS/" wp-config.php
14 sed -i "s/localhost/$DB_HOST/" wp-config.php
15
16 echo "=== Inyectando Salt Keys oficiales ==="
17 # Descargar claves aleatorias únicas de la API de WordPress
18 SALT_KEYS=$(curl -s https://api.wordpress.org/secret-key/1.1/salt/)
19
20 # Usamos un script rápido en PHP para reemplazar limpiamente el bloque de salts por seguridad
21 php <<ENDPHP
22 <?php
23 \$wpConfigPath = '/var/www/html/wp-config.php';
24 \$config = file_get_contents(\$wpConfigPath);
25
26 // Capturamos las claves descargadas pasadas por Bash
27 \$salts = <<<'SALTS'
28 $SALT_KEYS
29 SALTS;
30
31 // Expresión regular para ubicar y reemplazar el bloque completo por defecto
32 \$pattern = '/#@\\+\\.\\*?#@\\-\\/s';
33 \$config = preg_replace(\$pattern, \$salts, \$config);
34
35 file_put_contents(\$wpConfigPath, \$config);
36 ?>
37 ENDPHP
38
39 # Añadir configuraciones avanzadas al final del archivo
40 cat >> wp-config.php <<'EOF'
41
42 /* Configuración adicional de seguridad y optimización */
43 define('WP_DEBUG', false);
44 define('WP_AUTO_UPDATE_CORE', false);
45 define('DISALLOW_FILE_EDIT', true);
46
47 /* Dirección fija del sitio (IP privada del Web Server) */

```

Pruebas a realizar

```
# 1. Levantar infraestructura
vagrant up
```

```
C:\Users\Adrian\Documents\Computación en la nube y virtualización\PE02_wordpress>vagrant up
Bringing machine 'db' up with 'virtualbox' provider...
Bringing machine 'web' up with 'virtualbox' provider...
==> db: Importing base box 'ubuntu/focal64'...
==> db: Matching MAC address for NAT networking...
==> db: Checking if box 'ubuntu/focal64' version '20240821.0.1' is up to date...
==> db: There was a problem while downloading the metadata for your box
==> db: to check for updates. This is not an error, since it is usually due
==> db: to temporary network problems. This is just a warning. The problem
==> db: encountered was:
==> db:
==> db: Recv failure: Connection was reset
==> db:
==> db: If you want to check for box updates, verify your network connection
==> db: is valid and try again.
==> db: Setting the name of the VM: WP-Database
==> db: Clearing any previously set network interfaces...
==> db: Preparing network interfaces based on configuration...
db: Adapter 1: nat
db: Adapter 2: hostonly
==> db: Forwarding ports...
db: 22 (guest) => 2222 (host) (adapter 1)
==> db: Running 'pre-boot' VM customizations...
==> db: Booting VM...
==> db: Waiting for machine to boot. This may take a few minutes...
db: SSH address: 127.0.0.1:2222
db: SSH username: vagrant
db: SSH auth method: private key
db:
db: Vagrant insecure key detected. Vagrant will automatically replace
db: this with a newly generated keypair for better security.
db:
db: Inserting generated public key within guest...
db: Removing insecure key from the guest if it's present...
db: Key inserted! Disconnecting and reconnecting using new SSH key...
==> db: Machine booted and ready!
==> db: Checking for guest additions in VM...
db: The guest additions on this VM do not match the installed version of
db: VirtualBox! In most cases this is fine, but in rare cases it can
db: prevent things such as shared folders from working properly. If you see
db: shared folder errors, please make sure the guest additions within the
db: virtual machine match the version of VirtualBox you have installed on
db: your host and reload your VM.
db:
db: Guest Additions Version: 6.1.50
db: VirtualBox Version: 7.2
==> db: [vagrant-hostsupdater] Checking for host entries
==> db: [vagrant-hostsupdater] Writing the following entries to (C:/WINDOWS/system32/drivers/etc/hosts)
==> db: [vagrant-hostsupdater] 192.168.56.20 db-server # VAGRANT: 6898f36c6551da07895f905aa04d4688 (db) / d43adcc7-2335-4c92-ae46-415a389e51a9
==> db: Setting hostname...
```

```
# 2. Verificar que ambas VMs corren
vagrant status
```

```
C:\Users\Adrian\Documents\Computación en la nube y virtualización\PE02_wordpress>
C:\Users\Adrian\Documents\Computación en la nube y virtualización\PE02_wordpress>vagrant status
Current machine states:

db                               running (virtualbox)
web                               running (virtualbox)

This environment represents multiple VMs. The VMs are all listed
above with their current state. For more information about a specific
VM, run 'vagrant status NAME'.
```

```
# 3. Verificar conectividad entre VMs
vagrant ssh web -c "ping -c 3 192.168.56.20" vagrant
```

```
C:\Users\Adrian\Documents\Computación en la nube y virtualización\PE02_wordpress>vagrant ssh web -c "ping -c 3 192.168.56.20"
PING 192.168.56.20 (192.168.56.20) 56(84) bytes of data.
64 bytes from 192.168.56.20: icmp_seq=1 ttl=64 time=2.23 ms
64 bytes from 192.168.56.20: icmp_seq=2 ttl=64 time=1.02 ms
64 bytes from 192.168.56.20: icmp_seq=3 ttl=64 time=1.46 ms
```

```
ssh web -c "ping -c 3 db-server"
```

```
C:\Users\Adrian\Documents\Computación en la nube y virtualización\PE02_wordpress> vagrant ssh web -c "ping -c 3 db-server"

PING db-server (192.168.56.20) 56(84) bytes of data.
64 bytes from db-server (192.168.56.20): icmp_seq=1 ttl=64 time=0.630 ms
64 bytes from db-server (192.168.56.20): icmp_seq=2 ttl=64 time=0.620 ms
64 bytes from db-server (192.168.56.20): icmp_seq=3 ttl=64 time=1.31 ms

--- db-server ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2041ms
rtt min/avg/max/mdev = 0.620/0.853/1.311/0.323 ms
```

4. Verificar MySQL desde web

```
vagrant ssh web -c "mysql -h 192.168.56.20 -u wp_user -pwp_secure_pass -e 'SHOW DATABASES;'"
```

```
C:\Users\Adrian\Documents\Computación en la nube y virtualización\PE02_wordpress>vagrant ssh web -c "mysql -h 192.168.56.20 -u wp_user -pwp_secure_pass -e 'SHOW DATABASES;'"
mysql: [Warning] Using a password on the command line interface can be insecure.
+-----+
| Database |
+-----+
| information_schema |
| performance_schema |
| wordpress_db       |
+-----+
```

5. Verificar WordPress en navegador

http://localhost:8080 → Debería mostrar instalador de WordPress

Hola

¡Este es el famoso proceso de instalación de WordPress en cinco minutos! Simplemente completa la información siguiente y estarás a punto de usar la más enriquecedora y potente plataforma de publicación personal del mundo.

Información necesaria

Por favor, proporciona la siguiente información. No te preocupes, siempre podrás cambiar estos ajustes más tarde.

Título del sitio

Nombre de usuario

Los nombres de usuario pueden tener únicamente caracteres alfanuméricos, espacios, guiones bajos, guiones medios, puntos y el símbolo @.

Contraseña [Ocultar](#)

Fuerte

Importante: Necesitas esta contraseña para acceder. Por favor, guárdala en un lugar seguro.

Tu correo electrónico

Comprueba bien tu dirección de correo electrónico antes de continuar.

Visibilidad en los motores de búsqueda ☐ Pedir a los motores de búsqueda que no indexen este sitio

Depende de los motores de búsqueda atender esta petición o no.

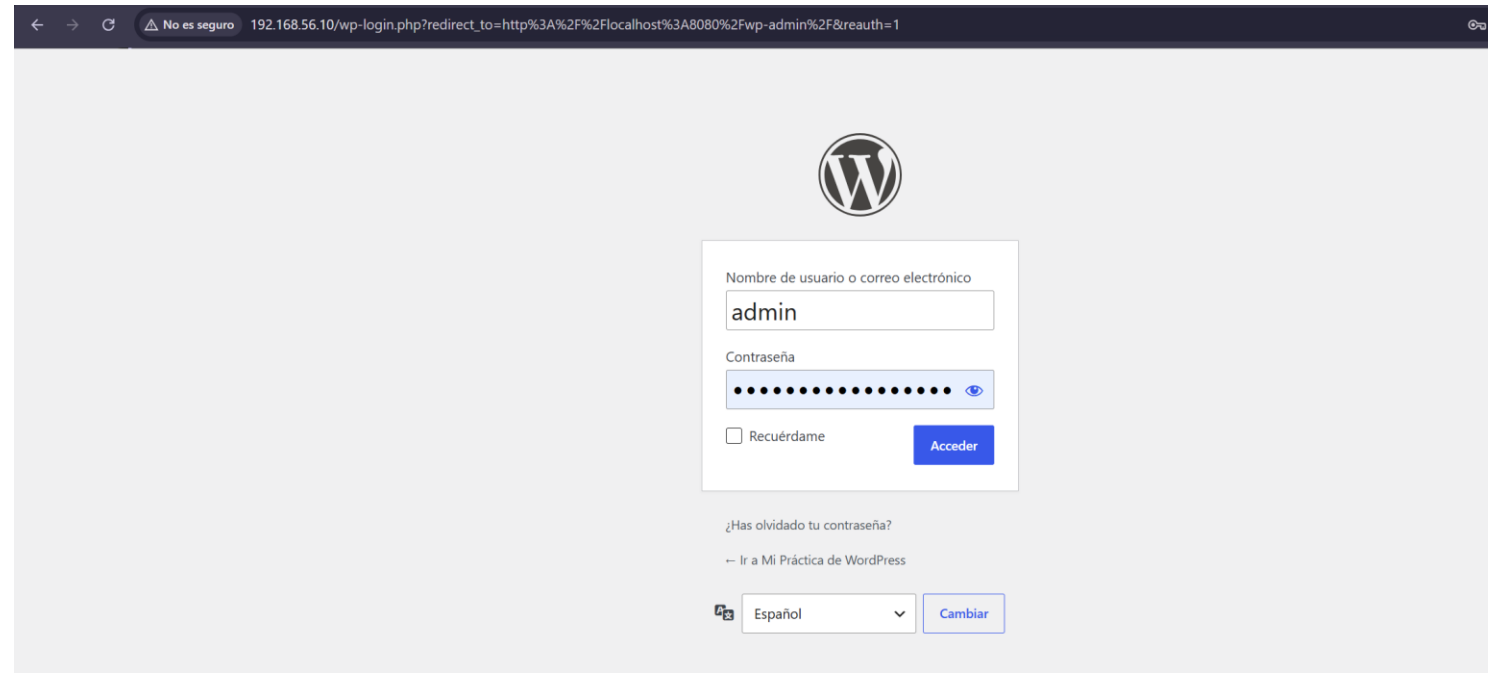
[Instalar WordPress](#)

6. Completar instalación de WordPress

- Título del sitio

- Usuario admin

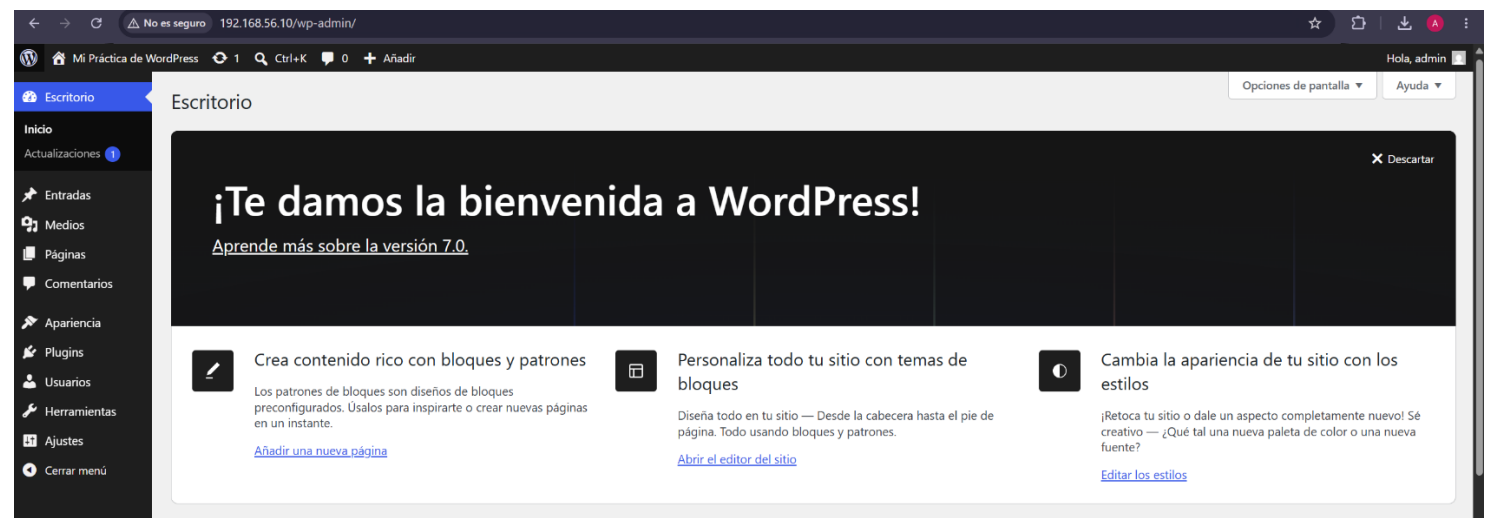
- Contraseña



- Contraseña
- Email



7. Verificar funcionamiento
http://localhost:8080/wp-admin



Troubleshooting

ySQL rechaza conexiones remotas

```
vagrant ssh db
sudo mysql -e "SELECT User, Host FROM mysql.user WHERE User='wp_user';" sudo
netstat -tlnp | grep 3306
sudo tail -f /var/log/mysql/error.log
```

```
vagrant@db-server:~$ vagrant ssh web
```

Command 'vagrant' not found, but can be installed with:

```
apt install vagrant
Please ask your administrator.
```

```
vagrant@db-server:~$ cat /var/www/html/wp-config.php | grep DB
```

```
cat: /var/www/html/wp-config.php: No such file or directory
```

```
vagrant@db-server:~$ php -r "new PDO('mysql:host=192.168.56.20;dbname=wordpress_db','wp_user','wp_secure_pass');"
PHP Fatal error: Uncaught PDOException: could not find driver in Command line code:1
```

Stack trace:

```
#0 Command line code(1): PDO->__construct()
```

```
#1 {main}
```

```
thrown in Command line code on line 1
```

```
vagrant@db-server:~$
```

```
vagrant@db-server:~$ sudo mysql -e "SELECT User, Host FROM mysql.user WHERE User='wp_user';"
```

User	Host
wp_user	192.168.56.%

```
vagrant@db-server:~$ sudo netstat -tlnp | grep 3306
```

```
tcp        0      0 0.0.0.0:3306 0.0.0.0:*        LISTEN      4279/mysqld
```

```
tcp        0      0 0.0.0.0:33060 0.0.0.0:*        LISTEN      4279/mysqld
```

```
vagrant@db-server:~$ sudo tail -f /var/log/mysql/error.log
```

```
2026-05-21T07:31:09.925298Z 0 [System] [MY-010931] [Server] /usr/sbin/mysqld: ready for connections. Version: '8.0.42-0ubuntu0.20.04.1' socket: '/var/run/mysqld/mysqld.sock' port: 3306 (Ubuntu).
2026-05-21T07:31:15.468588Z 0 [System] [MY-013172] [Server] Received SHUTDOWN from user <via user signal>. Shutting down mysqld (Version: 8.0.42-0ubuntu0.20.04.1).
2026-05-21T07:31:16.373871Z 0 [System] [MY-010910] [Server] /usr/sbin/mysqld: Shutdown complete (mysqld 8.0.42-0ubuntu0.20.04.1) (Ubuntu).
2026-05-21T07:31:16.754697Z 0 [System] [MY-010116] [Server] /usr/sbin/mysqld (mysqld 8.0.42-0ubuntu0.20.04.1) starting as process 4279
2026-05-21T07:31:16.763195Z 1 [System] [MY-013576] [InnoDB] InnoDB initialization has started.
2026-05-21T07:31:16.958505Z 1 [System] [MY-013577] [InnoDB] InnoDB initialization has ended.
2026-05-21T07:31:17.248147Z 0 [Warning] [MY-010068] [Server] CA certificate ca.pem is self signed.
2026-05-21T07:31:17.248178Z 0 [System] [MY-013602] [Server] Channel mysql_main configured to support TLS. Encrypted connections are now supported for this channel.
2026-05-21T07:31:17.271918Z 0 [System] [MY-011323] [Server] X Plugin ready for connections. Bind-address: '0.0.0.0' port: 33060, socket: /var/run/mysqld/mysqld.sock
2026-05-21T07:31:17.272085Z 0 [System] [MY-010931] [Server] /usr/sbin/mysqld: ready for connections. Version: '8.0.42-0ubuntu0.20.04.1' socket: '/var/run/mysqld/mysqld.sock' port: 3306 (Ubuntu).
```